Que corra solo sin perder el control

Agentes y automatización para quien no es programador: orquestar pasos, poner topes, programar corridas, auditar y cambiar sin romper.

Jonathan Rodríguez
Construir herramientas con IA, sin ser programador
linkedin.com/in/jonathanrod1986/

Antes de empezar

En las guías 01 y 02 diseñamos una herramienta que clasifica las reseñas de una tienda de café (ficticia) y escribe un informe mensual, con defensas para que la IA no invente. Funciona. El problema es que todavía depende de que alguien se acuerde de abrir el chat el primer día del mes, pegue el archivo y lance cada paso.

Esta guía es sobre quitar a esa persona del medio **sin perder el control**. Las dos cosas a la vez, porque automatizar sin control es la forma más rápida de que una herramienta haga un desastre a las seis de la mañana sin que nadie lo vea.

Primero, las palabras

Hoy todo el mundo habla de «agentes», y la palabra se usa para cosas muy distintas. Para esta guía:

- **Automatización:** pasos fijos que corren solos, en un orden que tú definiste. La IA puede estar en algunos pasos, pero el camino no lo decide ella.
- **Agente:** una IA que recibe un objetivo y unas herramientas (buscar, leer archivos, ejecutar algo) y **decide ella misma** qué pasos dar para llegar.

Nivel	Qué es	Ejemplo en la tienda	Cuándo usarlo
1	Prompt guardado	Instrucciones fijas que alguien pega cada mes	Tareas que se hacen poco
2	Flujo fijo con IA en algunos pasos	El informe mensual de las guías 01 y 02	La mayoría de lo que haces en marketing
3	Agente con herramientas, dentro de topes	Investigar qué dicen de la competencia antes de subir precios	Tareas donde el camino cambia cada vez
4	Varios agentes en paralelo con un crítico	Investigar competencia, precios y tendencias a la vez, y auditar el resultado	Investigación grande, con varios frentes

> **La regla que más plata ahorra:** usa el nivel más bajo que resuelva la tarea. Casi todo lo que la gente quiere hacer con «agentes» se resuelve mejor con un flujo fijo del nivel 2: es más barato, se puede predecir y, cuando falla, sabes dónde.

Qué te llevas

- Un orquestador corto en Python que encadena pasos, revisa que cada uno haya funcionado y deja un registro.
- Un semáforo para decidir qué hace la herramienta sola, qué hace y avisa y qué tiene que preguntarte.
- Código para topes de gasto, para evitar corridas dobles y para no procesar dos veces lo mismo.
- Las opciones para programar corridas, con lo que cuesta cada una.
- Plantillas de prompt para un agente de investigación y para un crítico.
- Un procedimiento para cambiar reglas sin romper lo que ya funciona.

Parte 1 • Del chat al proceso

El primer paso para automatizar no tiene nada de IA: es dejar de depender de una conversación. Mientras el proceso viva en un chat, depende de que alguien lo corra, de que se acuerde del orden y de que el chat no se pierda.

Cada paso, un programa con entrada y salida en archivo

En la guía 01 partimos la tarea en piezas. Ahora cada pieza se vuelve un programa pequeño que lee un archivo y escribe otro. La carpeta de la herramienta queda así:

```
tienda-resenas/
  orquestador.py        <- el que encadena todo
  pasos/
    recoger.py          lee la exportación y escribe datos/resenas.csv
    limpiar.py          quita datos personales: datos/resenas_limpias.csv
    clasificar.py       IA + validador: datos/clasificadas.jsonl
    revisar.py          filtros: datos/aprobadas.jsonl y datos/a_revision.jsonl
    contar.py           conteos: datos/conteos.json
    redactar.py         IA sobre los conteos: salidas/informe.md
    verificar.py        cifras y citas: salidas/informe_verificado.md
  datos/   salidas/   cache/   respaldo/
  registro.jsonl        <- qué pasó en cada corrida
```

¿Por qué separado y no un solo programa grande? Por la misma razón de la guía 01: si falla `redactar`, no vuelves a pagar la clasificación; corres desde `redactar` en adelante.

El orquestador

El orquestador es el que corre los pasos en orden. Lo importante no es que los corra: es que **revise que cada uno funcionó de verdad** antes de pasar al siguiente, y que deje anotado qué pasó.

```
import json, subprocess, sys, time, pathlib, datetime as dt

PASOS = [
    # (nombre, comando, archivo que el paso debe producir)
    ("recoger",    ["python3", "pasos/recoger.py"],    "datos/resenas.csv"),
    ("limpiar",    ["python3", "pasos/limpiar.py"],    "datos/resenas_limpias.csv"),
    ("clasificar", ["python3", "pasos/clasificar.py"], "datos/clasificadas.jsonl"),
    ("revisar",    ["python3", "pasos/revisar.py"],    "datos/aprobadas.jsonl"),
    ("contar",     ["python3", "pasos/contar.py"],     "datos/conteos.json"),
    ("redactar",   ["python3", "pasos/redactar.py"],   "salidas/informe.md"),
    ("verificar",  ["python3", "pasos/verificar.py"],  "salidas/informe_verificado.md"),
]
REGISTRO = pathlib.Path("registro.jsonl")

def anotar(**datos):
    datos["hora"] = dt.datetime.now().isoformat(timespec="seconds")
    with REGISTRO.open("a") as f:
        f.write(json.dumps(datos, ensure_ascii=False) + "\n")

def correr(desde=None):
    activo = desde is None
    for nombre, comando, salida in PASOS:
        activo = activo or nombre == desde
        if not activo:
            continue
        inicio = time.time()
        r = subprocess.run(comando, capture_output=True, text=True)
        archivo = pathlib.Path(salida)
        nuevo = archivo.exists() and archivo.stat().st_mtime >= inicio
        ok = r.returncode == 0 and nuevo
        anotar(paso=nombre, ok=ok, codigo=r.returncode,
               segundos=round(time.time() - inicio, 1), error=r.stderr[-500:])
        if not ok:
            print(f"Falló «{nombre}». Detalle en registro.jsonl", file=sys.stderr)
            sys.exit(1)
    print("Corrida completa. Falta la aprobación humana del informe.")

if __name__ == "__main__":
    correr(sys.argv[1] if len(sys.argv) > 1 else None)
```

Lo que hace el orquestador, en palabras

- **Corre cada paso** y espera a que termine.
- **Revisa dos cosas:** que el paso terminó sin error (código de salida 0) y que el archivo que debía producir **es nuevo**. Esto último es clave: el archivo del mes pasado sigue ahí, así que «existe» no prueba nada. Por eso compara la hora del archivo con la hora en que arrancó el paso.
- **Anota** en `registro.jsonl` cada paso: si funcionó, cuánto tardó y el final del mensaje de error.
- **Se detiene** en el primer paso que falle. Nunca sigue con datos viejos.
- **Puede arrancar desde un paso:** `python3 orquestador.py redactar` corre solo de la redacción en adelante.

Error que ya vi: el programa que se ejecuta solo

Fíjate en las dos últimas líneas: `if __name__ == "__main__":`. Significan «corre esto solo si alguien ejecutó este archivo directamente». Sin ellas, el código se ejecuta también cuando **otro** programa lo importa para usar una función suya.

Me pasó: un programa que reescribía unos inventarios se ejecutó solo, sin que nadie lo llamara, porque otro programa lo importó para usar una función pequeña. Desde entonces, todo programa que **cambia** algo (escribe, borra, envía, cobra) lleva esa guarda. Sin excepción.

Parte 2 • Topes y permisos

Automatizar es darle autonomía a la herramienta. La pregunta no es si confías en ella, es **hasta dónde**. Y eso se escribe antes, no se improvisa a las seis de la mañana.

El semáforo

Para cada cosa que la herramienta puede hacer, decide de qué color es:

Color	Qué significa	En la tienda
Verde	Lo hace sola, sin avisar	Leer reseñas, limpiar, clasificar dentro del tope, recalcular conteos, generar el borrador
Amarillo	Lo hace sola y te avisa	Segunda pasada de dudosas, reintentos, casos enviados a revisión, bloqueos de filtros
Rojo	No lo hace sin tu permiso	Enviar el informe, gastar por encima del tope, cambiar la lista de categorías, borrar datos, publicar algo

La regla es simple: **dentro de los topes, se opera sin pedir permiso**. Pedir permiso para todo vuelve inútil la automatización; no pedirlo nunca la vuelve peligrosa. El semáforo escrito es lo que permite las dos cosas: autonomía donde no hay riesgo, y freno donde sí lo hay.

> **Lo que siempre es rojo:** todo lo que sale de tu control y no se puede devolver. Enviar a un cliente, publicar, pagar, borrar sin respaldo. Aunque la herramienta lo haga perfecto 99 veces, la vez 100 te cuesta más de lo que ahorraron las otras 99.

El tope de gasto, en código

Un tope que solo está escrito en un documento no es un tope. Tiene que estar en el código y detener la corrida. El truco: **cobrar antes de llamar**, con el peor caso (la respuesta más larga posible), para que nunca te pases ni por una llamada.

```
class Presupuesto:
    def __init__(self, tope_usd, precio_entrada, precio_salida):
        self.tope = tope_usd
        self.gastado = 0.0
        self.pe = precio_entrada / 1_000_000   # por token
        self.ps = precio_salida / 1_000_000

    def autorizar(self, tokens_entrada, max_tokens_salida):
        peor_caso = tokens_entrada * self.pe + max_tokens_salida * self.ps
        if self.gastado + peor_caso > self.tope:
            raise RuntimeError(f"Tope de US${self.tope} alcanzado: la corrida se detiene")
        self.gastado += peor_caso
```

`clasificar.py` llama a `autorizar` antes de cada llamada a la IA. Si llegaron 3.000 reseñas en vez de 300, o si algo entró en un ciclo, la corrida se detiene sola y el registro dice por qué. Como se cobra el peor caso, el gasto real siempre queda un poco por debajo de lo que el presupuesto anota.

No procesar dos veces lo mismo

Si la corrida se cae a la mitad y la vuelves a lanzar, no quieres volver a clasificar (y pagar) las 150 reseñas que ya estaban listas. Para eso, cada paso revisa qué ya hizo:

```
import json, pathlib

hechas = pathlib.Path("datos/clasificadas.jsonl")
ya = set()
if hechas.exists():
    ya = {json.loads(linea)["id"] for linea in hechas.open()}
pendientes = [r for r in resenas if r["id"] not in ya]
```

A esto se le dice que el paso es **idempotente**: correrlo dos veces da el mismo resultado que correrlo una. Es lo que te deja relanzar sin miedo.

No correr dos veces al mismo tiempo

Si la corrida programada se demora y alguien la lanza a mano mientras tanto, dos procesos escriben el mismo archivo y lo dañan. Un candado sencillo lo evita:

```
import os, pathlib, sys

CANDADO = pathlib.Path("corriendo.lock")
if CANDADO.exists():
    sys.exit("Ya hay una corrida en marcha (o quedó un candado viejo: revísalo)")
CANDADO.write_text(str(os.getpid()))
try:
    correr()
finally:
    CANDADO.unlink()
```

Si el computador se apaga en plena corrida, el candado queda puesto y la siguiente no arranca. Es a propósito: prefiero que no corra y me entere, a que corra sobre una corrida rota.

Parte 3 • Programar que corra solo

Con el orquestador listo, falta que alguien lo lance. Estas son las opciones más comunes, de la más casera a la más robusta:

Opción	Dónde corre	Costo	Lo bueno	Lo que hay que saber
Tu computador (cron en Mac o Linux, Programador de tareas en Windows)	En tu máquina	Gratis	Simple, todo queda en tu disco	Solo corre si el computador está prendido
Tareas programadas de tu asistente de IA	En la app del asistente	Incluido en tu plan	La IA puede leer, escribir y revisar como lo harías tú	Depende de que la app esté abierta; revisa qué permite tu plan
GitHub Actions	En la nube	Gratis con límites de minutos al mes	Corre aunque tu computador esté apagado; guarda el historial	La hora va en UTC; las claves van como secretos, nunca en el código
Google Apps Script con activador por tiempo	En la nube de Google	Gratis con cuotas	Ideal si todo vive en Sheets y Drive	Tiene límites de tiempo por ejecución
Plataformas sin código (Make, Zapier, n8n)	En la nube o en tu servidor	Desde gratis, según volumen	Se arma arrastrando bloques	Los costos suben con el volumen; revisa cada plan

Los planes y límites cambian: revisa los vigentes antes de escoger.

Ejemplo con cron

Cron es el programador que viene con Mac y Linux. Esta línea corre el orquestador el día 1 de cada mes a las 6:00 a. m.:

```
0 6 1 * *  cd ~/tienda-resenas && python3 orquestador.py >> corridas.log 2>&1
```

Los cinco primeros campos son minuto, hora, día del mes, mes y día de la semana. `0 6 1 * *` se lee: «minuto 0, hora 6, día 1, cualquier mes, cualquier día de la semana».

Ejemplo con GitHub Actions

```
on:
  schedule:
    - cron: "0 11 1 * *"   # 11:00 UTC = 6:00 a. m. en Bogotá
jobs:
  informe:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: python3 orquestador.py
        env:
          CLAVE_IA: ${{ secrets.CLAVE_IA }}
```

Dos detalles que se olvidan: la hora del `cron` de GitHub va en **UTC** (Bogotá está cinco horas atrás), y la clave de la IA va como **secreto** del repositorio, nunca escrita en el archivo. Lo de las claves lo vemos a fondo en la guía 04.

Que la corrida te cuente qué pasó

Una corrida que termina en silencio es una corrida que no sabes si pasó. Al final, el orquestador debería mandarte (por correo, chat o una notificación) un resumen de cuatro líneas:

```
Informe de octubre: listo para aprobar
300 reseñas · 17 a revisión humana · 9 bloqueos de filtros
Gasto: US$1,32 de un tope de US$5
Urgencia alta: 24 (envío 15, cobro 9)
```

Y si falló, el resumen dice en qué paso y por qué. Ese mensaje es el que te deja dormir tranquilo.

> **Lo uso yo:** esta misma serie de guías y posts sale de un sistema así. Cada lunes, una tarea programada escoge las ideas de la semana de mi base de conocimiento, escribe los borradores con mi guía de voz, genera los PDF y pasa un filtro que bloquea lo que no puede salir. En mi semáforo, escribir borradores es verde; publicar es rojo. Yo leo, corrijo y publico.

Parte 4 • Agentes: cuándo sí y cómo

Hay tareas donde un flujo fijo no alcanza porque el camino cambia cada vez. Investigar es el mejor ejemplo: no sabes de antemano qué vas a encontrar ni qué fuente te va a llevar a la siguiente. Ahí sí sirve un agente.

Supón que la tienda va a subir precios y quiere saber, antes, qué está pasando afuera.

Un agente por frente

El error típico es mandar un solo agente a «investigar todo». Se dispersa, mezcla temas y te devuelve un documento largo donde no sabes de dónde salió cada cosa. Lo que funciona es **partir la investigación en frentes** y darle uno a cada agente:

Frente	Pregunta	Entrega
01 · Precios	¿A cuánto venden los competidores cafés comparables?	`docs/01-precios.md` + `datos/01-precios/`
02 · Reseñas de la competencia	¿De qué se quejan y qué elogian los clientes de la competencia?	`docs/02-competencia.md` + `datos/02-competencia/`
03 · Tendencias	¿Qué se está buscando y comentando sobre café de especialidad?	`docs/03-tendencias.md` + `datos/03-tendencias/`

Cada agente trabaja en su frente, con sus archivos, **sin ver lo que hacen los otros**. Así no se contaminan: si el agente de precios ve las tendencias, empieza a buscar precios de lo que está de moda y deja por fuera lo que de verdad compite con la tienda.

Cada cifra con su origen

La regla más útil para un agente de investigación: **cada cifra se marca según de dónde salió**.

Marca	Qué significa	Ejemplo
Verificada	Sale de una fuente primaria, con enlace y fecha	Precio tomado de la tienda del competidor, con fecha de consulta
Calculada	Sale de cifras verificadas, con la cuenta escrita	Promedio de precios de 8 competidores
Prensa o indicio	La dijo alguien y no se pudo confirmar en la fuente	«El consumo de café de especialidad crece», según una nota de prensa

Cuando lees el informe, sabes al instante cuánto pesa cada cifra. Y cuando alguien pregunta «¿de dónde sale eso?», la respuesta ya está escrita.

PLANTILLA · PROMPT DE UN AGENTE DE FRENTE

Rol: eres el investigador del frente [NN · nombre]. Solo trabajas este frente.

Meta que enmarca la investigación: [para qué es: p. ej. «decidir si subimos precios en noviembre»].

Pregunta: [la pregunta del frente].

Fuentes permitidas: [cuáles sí]. **Fuentes vetadas:** [cuáles no y por qué].

Entrega: escribe `docs/NN-tema.md` con hallazgos, cada uno con su fuente; guarda los datos en `datos/NN-tema/`. Marca cada cifra como verificada, calculada o de prensa.

Topes: máximo [N] búsquedas y [N] páginas leídas; si los alcanzas, entrega lo que tengas.

Entregas parciales: cada [N] hallazgos, guarda lo que llevas. Si te detienes, que no se pierda.

No hagas: no investigues otros frentes, no inventes cifras, no rellenes vacíos con supuestos sin marcarlos.

Dos detalles que ahorran corridas

- **Entregas parciales.** Los agentes a veces se cuelgan o se cortan a la mitad. Si solo guardan al final, pierdes todo. Que guarden a medida que avanzan.
- **Antes de relanzar, lee el registro.** Si una corrida se cortó, mira hasta dónde llegó y qué estaba haciendo. Relanzar a ciegas repite el gasto y, muchas veces, el mismo error.

La verificación del final

Cuando los frentes terminan, falta un paso: alguien tiene que intentar **tumbar** las conclusiones. Toma las cinco cifras o afirmaciones que más pesan en la decisión y pásaselas a un crítico (Parte 5) con una sola instrucción: «intenta demostrar que esto está mal». Lo que sobrevive, se usa.

Parte 5 • El crítico dentro de la cadena

En la guía 02 vimos el crítico revisando un informe. En un proceso automático cumple otra función: revisar **los cambios**. Mi regla es que **toda regla nueva pasa por un crítico** antes de quedarse.

Qué busca un buen crítico

Los errores más peligrosos de una herramienta no hacen ruido. Estos son los que más he visto pasar sin que nadie los note:

Error silencioso	Cómo se ve	Ejemplo en la tienda
Valor por defecto escondido	Un dato que falta se rellena solo con algo «razonable»	Las reseñas sin estrellas se cuentan como de 5
Conteo doble	Algo se suma en dos lugares	Una reseña con dos temas cuenta en los dos y el total da 340 de 300
Cifra sin denominador	Un porcentaje sin decir de qué total	«El 40 % se queja»: ¿de las 300 o de las 120 negativas?
Cambio que mueve lo que no debía	Arreglas una cosa y cambia otra	Ajustas envío y se mueve cobro

PLANTILLA · PROMPT DEL CRÍTICO DE CAMBIOS

Rol: eres un auditor adversarial. Tu trabajo es encontrar por qué este cambio está mal, no confirmar que está bien.

Contexto: [qué hace la herramienta, en tres líneas].

El cambio: [qué regla, prompt o lista cambió y por qué].

El antes y el después: [la tabla de diferencias de la Parte 6].

Revisa: valores por defecto escondidos, conteos dobles, cifras sin denominador, cambios en lo que no debía moverse y casos que la regla nueva deja por fuera.

Responde: publicable (sí o no), problemas encontrados con la evidencia de cada uno y qué revisarías primero.

Guarda cada auditoría

Cada revisión del crítico se guarda en un archivo con fecha (`auditorias/2026-11-02-cambio-envio.md`). En seis meses, cuando alguien pregunte por qué la herramienta cuenta así, la respuesta está ahí, con el antes, el después y lo que se revisó.

> **Un detalle práctico:** para revisar cosas visuales (gráficos, láminas, imágenes), me ha funcionado mejor armar una hoja de contactos con todas las imágenes en miniatura y revisarla yo, que mandar a un agente a abrirlas una por una.

Parte 6 • Cambiar sin romper

Llega el día en que hay que cambiar algo: la definición de una categoría, el prompt de redacción, el modelo. En un proceso automático, un cambio aguas arriba se propaga a todo lo de abajo. Por eso los cambios pasan por una **compuerta en cadena**: una secuencia fija de pasos que no se salta.

Los siete pasos

1. **Escribe la cadena.** Qué depende de qué: de la lista sale la clasificación; de ahí, los filtros, los conteos y el informe. Si cambias la lista, todo lo de abajo se regenera.
2. **Respalda.** Copia `datos/` y `salidas/` a `respaldo/AAAA-MM-DD/`, o, si usas git, marca la versión con una etiqueta. Si algo sale mal, vuelves atrás en un minuto.
3. **Ensaya sin tocar lo real.** Corre la cadena con el cambio en una carpeta aparte. Lo que ve el cliente (o el jefe) sigue intacto.
4. **Mide la diferencia.** Compara lo de antes con lo del ensayo.
5. **Aprueba.** Una persona mira la diferencia y decide.
6. **Regenera de verdad.**
7. **Que pase el crítico.** Sobre el resultado final, no sobre el ensayo.

Medir la diferencia

```
import json

def diferencias(antes, despues):
    viejo = json.load(open(antes))
    nuevo = json.load(open(despues))
    for clave in sorted(set(viejo) | set(nuevo)):
        a, b = viejo.get(clave, 0), nuevo.get(clave, 0)
        if a != b:
            print(f"{clave}: {a} -> {b} ({b - a:+})")

diferencias("datos/conteos.json", "ensayo/datos/conteos.json")
```

Supón que el cambio fue el de la guía 02: aclarar la frontera entre envío y atención. Lo esperado es esto:

Tema	Antes	Después	Diferencia
envio	93	99	+6
atencion	57	51	−6
producto, cobro, empaque	sin cambio	sin cambio	0

Lo que se movió es justo lo que tenía que moverse, y en la misma cantidad. Si en la tabla apareciera `cobro: 30 -> 27`, el cambio tocó algo que no debía: no se aprueba hasta entender por qué. Escribir antes de ensayar qué esperas que cambie es lo que convierte esta tabla en una prueba.

> **Regla de oro:** lo que ya vio el cliente no se regenera sin aprobación explícita. Puedes ensayar todo lo que quieras; publicar un número distinto del que ya entregaste es una conversación, no un paso automático.

Parte 7 • Que los errores se vean

El peor error de una automatización no es fallar: es **fallar en silencio**. El informe sale, se ve normal, y está hecho con datos del mes pasado porque un paso falló y nadie se enteró.

El código de salida

Todo programa, al terminar, devuelve un número: 0 si salió bien y otro si salió mal. Es la forma en que un programa le dice a otro «fallé». El orquestador de la Parte 1 lo revisa en cada paso (`r.returncode == 0`). Ese número **nunca se esconde**.

Error que ya vi: el filtro que tapó el error

Me pasó: un cambio rompió un paso de la cadena, pero la salida de ese paso pasaba por otro comando que mostraba solo las últimas líneas. El código de salida que llegó al final fue el del segundo comando, que sí había funcionado. La cadena siguió como si nada.

En la terminal de Mac o Linux se ve así:

```
python3 orquestador.py | tail -5        # si el orquestador falla, igual "sale bien"

set -o pipefail
python3 orquestador.py | tail -5        # ahora, si el orquestador falla, falla todo
```

Sin `pipefail`, el resultado de una cadena con `|` es el del **último** comando. Con `pipefail`, si cualquiera falla, falla todo. Va al principio de cualquier script que encadene comandos.

Tres reglas para que nada falle callado

- **Cada paso revisa lo que recibió.** Si `contar.py` recibe un archivo vacío, falla con un mensaje claro; no produce un informe con ceros.
- **Nada de `try` que se traga el error.** Atrapar un error para anotarlo y detenerse, sí. Atraparlo para seguir como si nada, nunca.
- **El resumen final dice lo que falló.** Si hubo bloqueos, reintentos o casos a revisión, aparecen en el mensaje, aunque la corrida haya terminado.

Todo junto: la tienda en piloto automático

Momento	Qué pasa	Color
Día 1, 6:00 a. m.	El programador lanza el orquestador; el candado evita corridas dobles	Verde
6:00 a 6:20	Recoger, limpiar, clasificar con tope de US$5, filtros, segunda pasada	Verde
6:20	17 casos a revisión humana, 9 bloqueos de filtros	Amarillo: avisa
6:25	Conteos, redacción, verificación de cifras y citas, crítico	Verde
6:26	Llega el resumen de cuatro líneas: «listo para aprobar»	Amarillo: avisa
Día 1, en la mañana	Una persona revisa los 17 casos (20 minutos) y lee el informe	Persona
Día 1	Aprobación y envío del informe	Rojo: pide permiso
Cuando haya un cambio	Compuerta en cadena: respaldo, ensayo, diferencia, aprobación, regeneración, crítico	Rojo

El trabajo humano bajó de 10 horas al mes a unos 40 minutos. Y lo más importante: esos 40 minutos son **decisiones**, no lectura.

Errores que ya vi

Error	Cómo se ve	Qué hacer
Un agente para todo	Caro, impredecible, difícil de depurar	Usar el nivel más bajo que resuelva la tarea
El proceso vive en un chat	Depende de que alguien se acuerde	Pasos en programas con archivos de entrada y salida
Revisar que el archivo existe	Se usa el archivo del mes pasado	Revisar que el archivo es nuevo
Programa sin guarda de ejecución	Se ejecuta solo al importarlo	`if __name__ == "__main__":` en todo lo que cambia algo
Tope solo en un documento	La corrida se pasa y nadie la detiene	Tope en código, cobrando el peor caso antes de llamar
Relanzar desde cero	Se paga dos veces lo mismo	Pasos idempotentes
Dos corridas a la vez	Archivos dañados	Candado
Un agente investigando todo	Temas mezclados, cifras sin origen	Un agente por frente, cifras marcadas por origen
Relanzar a ciegas	Mismo gasto y mismo error	Leer el registro antes
Cambiar directo en lo real	Cifras que se mueven sin explicación	Compuerta: respaldo, ensayo, diferencia, aprobación
Error escondido en una tubería	La cadena sigue con un paso roto	`set -o pipefail` y códigos de salida revisados
Corrida silenciosa	No sabes si pasó	Resumen al final, también cuando falla

Lista de verificación

- ☐ Escogí el nivel más bajo que resuelve la tarea (flujo fijo antes que agente).
- ☐ Cada paso es un programa con un archivo de entrada y uno de salida.
- ☐ El orquestador revisa el código de salida y que el archivo producido sea nuevo.
- ☐ Hay un registro de cada corrida, paso por paso.
- ☐ Todo programa que cambia algo tiene guarda de ejecución.
- ☐ Escribí el semáforo: verde, amarillo y rojo.

- ☐ Enviar, publicar, pagar y borrar son rojos.
- ☐ El tope de gasto está en el código y detiene la corrida.
- ☐ Los pasos son idempotentes: relanzar no duplica ni vuelve a pagar.
- ☐ Hay un candado contra corridas simultáneas.
- ☐ Escogí dónde se programa la corrida y sé qué pasa si la máquina o la app están apagadas.
- ☐ Las claves están como secretos, nunca en el código.

- ☐ Al final de cada corrida llega un resumen, también cuando falla.
- ☐ Si uso agentes: uno por frente, con su meta, sus fuentes, sus topes y entregas parciales.
- ☐ Cada cifra de una investigación está marcada como verificada, calculada o de prensa.
- ☐ Las conclusiones importantes pasaron por un crítico que intentó tumbarlas.
- ☐ Toda regla nueva pasa por el crítico y la auditoría queda guardada.
- ☐ Los cambios pasan por la compuerta: respaldo, ensayo, diferencia, aprobación, regeneración y crítico.

- ☐ Escribí qué esperaba que cambiara antes de mirar la diferencia.
- ☐ Ningún error se esconde: `pipefail`, códigos de salida y nada de errores tragados.

De dónde sale esto

Todo lo de esta guía lo he aplicado en procesos reales: investigaciones con varios frentes en paralelo, cadenas de datos que se regeneran, motores que corren dentro de topes de gasto y esta misma serie de contenidos, que sale de una tarea programada cada lunes. El ejemplo de la tienda de café y sus números son **ficticios**. Los dos errores que cuento (el programa que se ejecutó al importarlo y el filtro que tapó un error) me pasaron de verdad.

No soy programador; vengo de la estrategia. El código de esta guía es corto a propósito, para que se entienda. Si alguien de ingeniería lo haría distinto, me sirve que me lo diga.

En la próxima guía: entregar y mantener. Cuánto cuesta de verdad una herramienta en el tiempo, cómo manejar claves sin filtrarlas, cómo publicar sin pisar lo publicado y cómo auditar una herramienta que ya está andando.

Si esta guía te sirvió, guárdala y pásasela a alguien que todavía corre su proceso a mano cada mes.